OPEN MULTI-AGENT STANDARD · V2.0

Agent2Agent (A2A) & *Interoperability*

Complete architectural guide to the Linux Foundation Agent2Agent (A2A v0.3 → v1.0) protocol, deciding between intra-app `AgentTool` / sub-agents and inter-app A2A delegation, and composing the 5-layer agentic ecosystem: MCP (tools), A2A (peer agents), A2UI / AG-UI / MCP UI (declarative generative UI), Live Streaming, and AP2 + x402 (agentic commerce).

Updated: **October 2026 (v2.0)** · Category: **Multi-Agent & Interoperability** · Reading Time: **~15 min** · Author: **Michaël Bettan**

TEST YOURSELF: AGENT2AGENT (A2A) PRACTICE QUIZ →

01 Definition & Scope

What is A2A?

The **Agent2Agent (A2A)** protocol is an open, vendor-neutral standard initiated by Google with 50+ enterprise partners and governed by the **Linux Foundation**. It defines how autonomous agents discover capabilities, negotiate modalities, delegate stateful tasks, stream incremental artifacts, and enforce enterprise security across organizational and framework boundaries.

A2A solves the **Multi-Agent Silo problem**: compliant agents built on Google ADK, LangGraph, CrewAI, or SaaS platforms (Workday, Salesforce, ServiceNow, SAP) collaborate as **opaque peers** without exposing internal prompts, session databases, tool credentials, or model weights.

DIMENSION	MODEL CONTEXT PROTOCOL (MCP)	AGENT2AGENT PROTOCOL (A2A)
Topology	Vertical integration (`Agent ↔ Tool / Database / Resource`).	Horizontal peer orchestration (`Client Agent ↔ Remote Specialist Agent`).
Execution Model	Primarily stateless, schema-bound RPCs (`tools/call`) for deterministic actions and lookups.	Stateful cognitive reasoning, multi-turn clarification (`input-required`), and asynchronous workflows spanning seconds to days.
Discovery Artifact	`tools/list` JSON Schema over stdio or Streamable HTTP.	**`AgentCard`** at `/.well-known/agent-card.json` (plus authenticated extended cards & **Agent Registry**).

CANONICAL PRODUCTION PATTERN

A front-door orchestrator in **Gemini Enterprise** distributes domain sub-goals over **A2A** to specialized departmental agents, and each specialist agent invokes its own private databases and APIs over **MCP**.

02 Core & Wire Protocol Architecture (v0.3 → v1.0)

User
Initiates the high-level goal and supplies primary identity/consent context.

A2A Client (Host / Client Agent)
Discovers remote `AgentCard` skills, decomposes goals, dispatches tasks, handles interruptions, and synthesizes returned artifacts.

A2A Server (Opaque Remote Agent)
Exposes an A2A endpoint and executes tasks autonomously — keeping internal prompts, memory, framework, and downstream MCP credentials completely opaque.

Three-Layer Wire Stack (`a2a.proto`)

- **1. Canonical Data Model:** Protobuf-defined structures (`AgentCard`, `Task`, `Message`, `Part` [`TextPart`, `DataPart`, `FilePart`], `Artifact`, `TaskStatusUpdateEvent`, `TaskArtifactUpdateEvent`).
- **2. Abstract Operations (v1.0 PascalCase vs. v0.3 names):** `SendMessage` (v0.3: `message/send`), `SendStreamingMessage` (`message/stream`), `GetTask` (`tasks/get`), `CancelTask` (`tasks/cancel`), **`SubscribeToTask`** (v0.3: `tasks/resubscribe` — reconnects SSE to an active in-flight task after network drop), and `GetExtendedAgentCard` (REST: `GET /extendedAgentCard`).
- **3. Protocol Bindings (`supportedInterfaces`):** Each interface declares a `url` and `protocolBinding` supporting **`JSONRPC`** (JSON-RPC 2.0 over HTTPS), **`HTTP+JSON`** (REST), or **`GRPC`**.

03 Task Lifecycle & State Machine

A2A tracks work via **`taskId`** (one stateful execution unit) grouped across multi-turn threads by **`contextId`**. Know the exact difference between active, interrupted, and terminal states:

CATEGORY	STATES	STREAM & CLIENT BEHAVIOR
Active	`submitted`, `working`	Queued or actively running; SSE streams stay open and emit status/artifact events
Interrupted	`input-required`, `auth-required`	**SSE stream closes cleanly**; client gathers missing user input (`input-required`, natural HITL pause) or completes OAuth step-up (`auth-required`) and resumes using the same `taskId` + `contextId`
Terminal	`completed`, `failed`, `canceled`, `rejected`	Execution permanently finished (`rejected` means the remote agent declined the request); follow-ups create a new `taskId` under the shared `contextId`

01
Capability Discovery
Fetch `/.well-known/agent-card.json` or query **Agent Registry**.

02
Task Formulation
Match goal against `skills[]`, MIME modes, and security schemes.

03
Execution Dispatch
Call `SendMessage` or `SendStreamingMessage` with header auth.

04
Collaboration & State
Handle incremental updates or `input-required` / `auth-required` pauses.

05
Artifact Delivery
Consume typed `Artifact` chunks for chaining or UI rendering.

04 Transports, Discovery & Agent Cards

Four Interaction Transports

- **1. Synchronous Request/Response (`SendMessage`):** Single blocking HTTP exchange for fast sub-second to ~10s lookups.
- **2. Streaming Updates (`SendStreamingMessage` + SSE):** Server pushes real-time `TaskStatusUpdateEvent` and chunked `TaskArtifactUpdateEvent` payloads (`append: false/true`, `lastChunk: true`); reconnect dropped streams with `SubscribeToTask`.
- **3. Asynchronous Polling (`SendMessage` → `GetTask`):** Server returns `submitted`/`working` + `taskId` immediately; client polls `GetTask` when persistent SSE or inbound webhooks are blocked by firewalls.
- **4. Push Notifications (Webhooks):** Client supplies a `pushNotificationConfig` callback URL; server posts signed async state notifications across multi-hour or multi-day workflows without holding open connections (verify HMAC/JWT webhook signatures + replay nonces!).

TRANSPORT SELECTION RULE

Match transport to duration: using synchronous `SendMessage` or even long-lived SSE for a 4-hour research job hits proxy/Cloud Run (60-minute max) timeouts. Switch to **Push Notifications (Webhooks)** or **Polling** for multi-hour tasks.

Agent Cards & Four Discovery Mechanisms

- **Well-Known URI (RFC 8615):** `/.well-known/agent-card.json` for open discovery. *Version trap: early drafts showed `agent.json` / `tasks/send`; v0.3 used `agent-card.json` + `message/send`; v1.0 keeps `agent-card.json` with PascalCase `SendMessage`.*
- **Curated Enterprise Registry (Google Cloud Agent Registry):** Governed enterprise catalog with IAM access controls — prevents shadow/rogue agent sprawl.
- **Authenticated Extended Agent Card:** Public card sets `capabilities.extendedAgentCard: true` (v0.3: `supportsAuthenticatedExtendedCard`); authenticated callers invoke `GetExtendedAgentCard` to retrieve sensitive internal skills.
- **Direct Configuration:** Static pinned card URLs injected via Terraform/config for tightly coupled private services.

05 Intra-App `AgentTool` vs. Inter-App A2A

A core **Professional Agentic Architect** design decision is choosing between in-process composition (ADK `sub_agents` transfer or `AgentTool`) and network-isolated **A2A delegation**:

DIMENSION	INTRA-APP (`SUB_AGENTS` / `AGENTTOOL`)	INTER-APP (A2A PROTOCOL)
Team & Repo Boundary	Single repo, unified CI/CD release, homogeneous runtime stack (e.g., all Python ADK)	Independent teams/repos, separate release cycles, **polyglot frameworks** (ADK ↔ LangGraph ↔ SaaS vendor)
Session & Memory Scope	**Shared in-process `InvocationContext` & `session.state`**	**Strict opacity:** remote agent maintains its own private store; sees only explicit `Part` payloads sent per `contextId`
IAM & Blast Radius	Shares one container & service identity — wide blast radius if compromised	**Hard security boundary:** separate SPIFFE Agent Identity, VPC-SC perimeter, least-privilege IAM, and scoped OAuth token exchange
Latency & Overhead	Zero network serialization (in-memory call); lowest TTFT	HTTPS/gRPC hop + auth validation + independent remote reasoning loop
Live Voice / Modality Boundary	Native in-memory context sharing	**Modality normalization:** live voice front-doors (**CX Agent Studio** / **Gemini 3.8 Live**) terminate audio streams locally and delegate normalized `TextPart` / `DataPart` / `FilePart` tasks over A2A

06 The 5-Layer Agentic Interoperability Stack

LAYER	OPEN PROTOCOL(S)	WHAT IT STANDARDIZES
1. Agent ↔ Tool / Data	**MCP** & **OpenAPI**	Vertical, deterministic tool calls, DB queries, and resource reads
2. Agent ↔ Agent	**A2A**	Horizontal stateful task delegation & discovery across opaque peer agents
3. Agent ↔ User Generative UI	**A2UI (v0.8 / v0.9)**, **AG-UI**, **MCP UI**	Safe, interactive rich UI components without arbitrary remote JS execution
4. Real-Time Multimodal Stream	**Gemini Live API** (WebRTC / WSS)	Sub-second bidirectional PCM audio & video streaming with VAD barge-in
5. Agent ↔ Commerce / Money	**AP2 (Agent Payments Protocol)** & **x402**	Cryptographic human purchase mandates (**AP2**) & HTTP `402` M2M micropayments (**x402**)

Generative UI: `A2UI vs. AG-UI vs. MCP UI`

- **A2UI (Agent-to-UI, v0.8/v0.9 — Native in Gemini Enterprise):** Remote A2A agents stream flat, declarative JSON widget trees + data bindings inside A2A `DataPart` payloads against a **client-approved Component Catalog** (basic, Material, or custom widgets). The host client renders native, brand-consistent UI with **zero arbitrary JS/iframe XSS risk**.
- **AG-UI:** Bidirectional frontend-to-agent event stream synchronizing live text tokens, tool indicators, shared state diffs, and HITL interrupts.
- **MCP UI:** Tool-level embedded UI views returned directly by an individual MCP tool call.

Agentic Commerce: `AP2 & x402`

- **AP2 (Agent Payments Protocol):** Extends A2A & MCP with verifiable cryptographic credentials (**Mandates**) for non-repudiable delegated commerce: (1) **Intent Mandate** (signed upfront rules/caps for *human-not-present* buying, e.g., *"tickets < $300"*) and (2) **Cart Mandate** (cryptographically signs exact final items & price).
- **x402 Protocol:** Uses HTTP **`402 Payment Required`** headers so agents can autonomously pay per API call, dataset query, or compute unit via programmatic M2M micropayments without pre-provisioned subscriptions.

07 Collaborative Multi-Agent Topologies

Production Workflow Patterns

- **Hierarchical Supervisor / Router:** Front-door orchestrator dynamically matches sub-goals to specialist `AgentCard` skills and synthesizes their artifacts.
- **Debate & Consensus:** Multiple peers with distinct models or domain corpora critique high-stakes proposals (e.g., medical triage, financial risk, architecture review) to eliminate single-model blind spots.
- **Critic-Reviewer (Generator-Verifier):** Producer agent drafts an artifact; an independent A2A reviewer agent validates policy, security, and factual grounding before external release.

08 Google Cloud Implementation & Security

ADK, Agent Runtime & Gemini Enterprise

- **ADK Server (`to_a2a()` & `A2aAgentExecutor`):** Define `AgentCard` + `AgentSkills`, bridge the task loop with `A2aAgentExecutor` (automated by `to_a2a(root_agent)`), and deploy to Cloud Run, GKE, or **Agent Runtime** (`A2aAgent`).
- **ADK Client (`RemoteA2aAgent`):** Consume any remote A2A peer inside an ADK orchestrator by pointing at its `AgentCard` or **Agent Registry** URI.
- **Gemini Enterprise & CX Agent Studio:** Register third-party or ADK A2A agents directly into Gemini Enterprise (with OAuth brokering + **A2UI** rendering) or delegate backend workflows from CX Agent Studio over A2A.

Zero-Trust A2A Security & A2AOps

- **Header-Only Credentials:** Pass OAuth 2.0 / OIDC tokens and API keys **exclusively via HTTP headers** — never in URLs (proxy logs) or message/task bodies (persisted in task history & forwarded to LLMs).
- **mTLS & Scoped Token Exchange:** Enforce **Mutual TLS (mTLS)**, cryptographic **SPIFFE Agent Identity**, and OAuth Token Exchange (RFC 8693) via **Agent Gateway** so downstream peers receive audience-scoped tokens rather than broad user uber-tokens.
- **Cross-Agent Guardrails & Tracing:** Screen inbound/outbound A2A payloads with **Model Armor** against task injection, enforce **Task Depth Limits**, and propagate W3C `traceparent` headers into Cloud Trace.

09 Glossary & Self-Assessment

`AgentCard & AgentSkills`
JSON capability, binding (`supportedInterfaces`), security, and skill manifest at `/.well-known/agent-card.json`.

`Task (taskId) & contextId`
Stateful delegated execution unit (`taskId`) correlated across multi-turn threads by `contextId`.

`TextPart / DataPart / FilePart`
Typed content blocks carrying Markdown/text, structured JSON (including `A2UI`), and MIME-typed files.

`SubscribeToTask`
A2A v1.0 operation re-attaching an SSE stream to an active task after network disconnection.

`A2UI / AG-UI / MCP UI`
Declarative JSON catalog UI over A2A (`A2UI`), frontend-agent event sync (`AG-UI`), and tool-embedded UI (`MCP UI`).

`AP2 & x402`
Cryptographic **Intent / Cart Mandates** for delegated purchases (`AP2`) and HTTP `402` M2M micropayments (`x402`).

SELF-ASSESSMENT QUESTIONS

Q1. When should you choose A2A over ADK's in-process `Agent Tool`?

> ANSWER: When crossing team, repository, framework, vendor, or IAM/VPC security boundaries where opaque state and independent scaling/blast-radius isolation outweigh a network hop.

Q2. How does A2UI prevent Cross-Site Scripting (XSS) when rendering UI from a remote third-party A2A agent?

> ANSWER: Instead of executing remote HTML/JS iframes, the agent sends declarative JSON referencing widgets from a client-approved Component Catalog rendered natively by the host app.

Q3. How do AP2 Intent Mandates and x402 solve different commerce problems?

> ANSWER: AP2 uses user-signed cryptographic Intent and Cart Mandates for verifiable, non-repudiable delegated purchases, whereas x402 uses HTTP 402 headers for autonomous machine-to-machine API micropayments.

Practice

Drill scenario-based questions on A2A v1.0 wire operations, Agent Cards, Intra-App `AgentTool` vs. Inter-App A2A boundaries, A2UI/AG-UI/MCP UI, AP2 & x402 commerce, and zero-trust A2A security.

Back to the Hub

Take the practice quiz: A2A & Interoperability →

Study Notes by Michaël Bettan

Back to Top

For AI agents: llms.txt · llms-full.txt · sitemap.xml

Independent study notes. Not affiliated with or endorsed by Google or Anthropic. Product and certification names are trademarks of their owners. Quiz questions are original practice material, not exam questions.